JOMO KENYATTA UNIVERSITY
OF
AGRICULTURE & TECHNOLOGY

SCHOOL OF OPEN, DISTANCE AND eLEARNING
P.O. Box 62000, 00200
Nairobi, Kenya
E-mail: elearning@jkuat.ac.ke

BIT 2114 Object Oriented Analysis and Design

LAST REVISION ON April 28, 2014

This presentation is intended to covered within one week. The notes, examples and exercises should be supplemented with a good textbook. Most of the exercises have solutions/answers appearing elsewhere and accessible by clicking the green Exercise tag. To move back to the same page click the same tag appearing at the end of the solution/answer.

Errors and omissions in these notes are entirely the responsibility of the author who should only be contacted through the **Department of Curricula & Delivery (SODeL)** and suggested corrections may be e-mailed to elearning@jkuat.ac.ke.

LESSON 9

Other UML Diagrams

Learning outcomes

Upon completing this topic, you should be able to:

1. Describe state transition, activity and implementation diagrams
2. Model the requirements from the use case diagram to all of the models examined.

9.1. Development of Dynamic Behavior

Use cases suggest the system events which are explicitly shown in system interaction diagrams. An initial, textual description of the system events and their goal is presented by operation contracts. The system events represent messages that initiate interaction diagrams, which illustrate how objects interact to fulfill the required tasks.

9.1.1. Dynamic Behavior: Perspective

The dynamic behavior of a use case viewed from the conceptual perspective shows:

- actors of a use case,
- interactions of the actors with the system

The implementation perspective of a use case shows

- the requests from outside (i.e., system operations),
- the internal messages and system events,
- the system as structured components (e.g., classes) involved in the process.

9.1.2. Use Cases and Interaction Diagrams

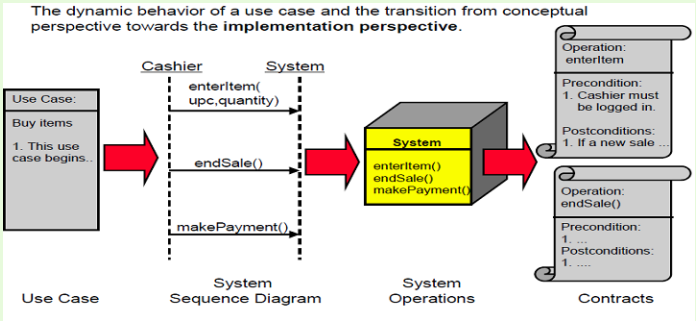

Figure 9.1: Use Cases and Interaction Diagrams

9.1.3. Use Case and System Interaction

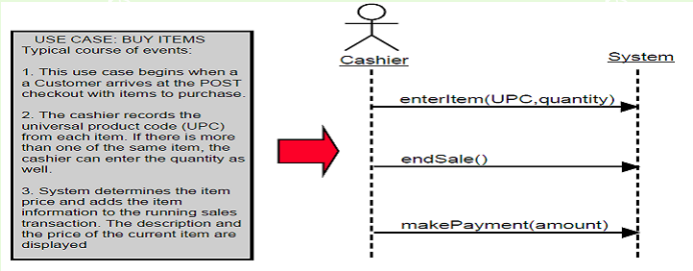

Figure 9.2: Use Cases and Interaction Diagrams

9.1.4. Contracts

The system sequence diagram shows the system events that an external actor generates. It does not elaborate on the details

of the functionality associated with the system operations invoked. In general a contract is a document that describes what an operation commits to achieve. A system operation contract describes changes in the state of the overall system when a system operation is invoked.

9.1.5. Contracts and Collaboration Diagrams

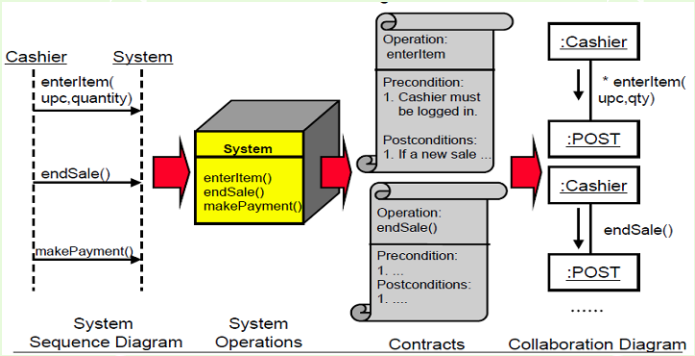

Figure 9.3: Contracts and Collaboration Diagrams

9.2. State Transition Diagram

Sequence and collaboration diagrams are ways of representing all the iterations that take place between objects involved in a

particular use case. Sometimes, however, we need to focus on specific class to illustrate how its objects behave during their lifetimes and how they react to all use cases in which they are involved. In this case, the suitable diagram is called the state diagram.

Definition:

A state diagram can also be defined as a model of the state of an object and the events that cause the object to change from one state to another. A state diagram describes the behavior of a single class. It describes hoe a class of objects behave in the system. In other words, it describes how a class responds to all use cases that affect it.

If a class always responds exactly in the same way to events

then there's no point of drawing a state diagram for it. The interest is in classes where the response of an object of the class to a particular event varies depending on the state the object is in at the time. So when you want to find out what's going on within a class, you need a state diagram.

A state transition diagram shows the states of a single object, the events or the messages that cause a transition from one state to another and the action that result from a state change. A state transition diagram will not be created for every class in the system.

9.2.1. Components of State Diagram

1. Start State
2. Stop state

3. State Transition

9.2.2. Semantics of Every Component

1. State:

- A state is a condition during the life of an object when it satisfies some condition, performs some action, or waits for an event.
- It represents a period of time during which the object satisfies some conditions for some events.
- The UML notation for a state is a rectangle with rounded corners.

1. Special states: There are two special states.
    - Start state: Each state diagram must have one and

only one start state. Notation for start state is "filled solid circle".

- Stop State: An object can have multiple stop states. Notation for stop state is bull's eye.

2. Event

- This is something that takes place at a certain period in time.
- It is an occurrence that triggers a state transition.
- Some examples of events are: a customer places some order, a student registers for a class, a person applies for a loan, a company hires a new employee.

1. State transition:

- A state transition represents a change from an originating

to a successor state.

- This represents the response of an object to an event.
- The response may involve movement of the object from one state to another or the object remaining the same state (sometimes referred to as self transition).
- The state transition changes in the attributes of an object or in the links and objects as with other objects. The state of an object depends in its attribute values and links to other objects.
- An object remains in a particular state for sometime before transitioning to another state
  - For example an employee object might be in the part time state (as specified in the employee status at-

tribute) for a few months before transitioning to a full time state based on a recommendation from the manager (event).

- A state transition is regarded as instantaneous and cannot be interrupted.
- It consists of three parts: the event, guard and action. All these three parts are optional
- An event triggers a state transition.
- A guard is a condition that allows the transition to take place only if the condition is true. Guards are written inside square brackets [].
- An action is the behavior that occurs when a transition takes place and is presided by a slash (/).

- Both guards and actions are generally implemented as operations on the class in the final system code.
- Transition label: event name[guard condition] / action

9.2.3. Notations

1. Initial state: A filled circle represents the object initial state.
2. Final state: A filled circle nested inside another circle. A bull's eye represents the object final state.
3. State: A rectangle with round corners represents a situation during the life of an object.
4. Transition arrow: This represents the path between different states of an object. The transition is labeled with the

event that triggered it and the action that results from it.

Note:

Transition events are named in the past tense because they have already occurred within the transition. State chart diagrams are not created for all classes. They are created when:

1. A full class has a complex lifecycle.
2. An instance of class updates its attributes through the lifecycle.
3. A class has an operational life cycle.
4. Two classes depend on each other.
5. The object's current behavior depends on what happened previously.

9.2.4. Steps for Building State Chart Diagram

1. Set the context: the context of a state chart diagram is usually a class. However, it also could be a set of classes, a subsystem, or an entire system.
2. Identify initial, final and stable states of the object. This involves identifying the various states that an order will have over its lifetime. This includes establishing the boundaries of the existence of an object, by identifying the initial and final states of an object. We must also identify the stable state of an object. The information necessary to perform this step is reading from the use case descriptions, talking with users and relying on the requirement gathering technique. An easy way to identify the states

of an object is to write the steps of what happens to an object over time from start to finish.

3. Determine the order in which the object will pass through the stable state. Here we determine the sequence of state that an object will pass through during this lifetime. Using this sequence, we can place the states on the state diagrams in a left to right order.

4. Identify the events, actions and guard conditions associated with the transitions. Events actions and guard conditions associated with the transition between the states of the object are identified and shown on diagram. The events are the triggers that cause an object to move from one state to the next state. In other words, an event causes an action to execute that changes the value (s) of an ob-

ject attribute (s) in a significant manner. The actions are typically operations contained within the object. Guard conditions can model a set of test condition that must be met for the transition to occur. At this point in the process, the transition are drawn between the relevant states and labeled with the event, action or guard condition.

5. Validate the state diagram. The state diagram is validated by making sure that each state is reachable and that is possible to leave all state except for final state. If an identified state is not reachable either a transition is missing or the state was identified in error. Furthermore, only final state can be a dead end from the perspective of an object lifecycle.

9.2.5. State Transition Diagram for Account Class

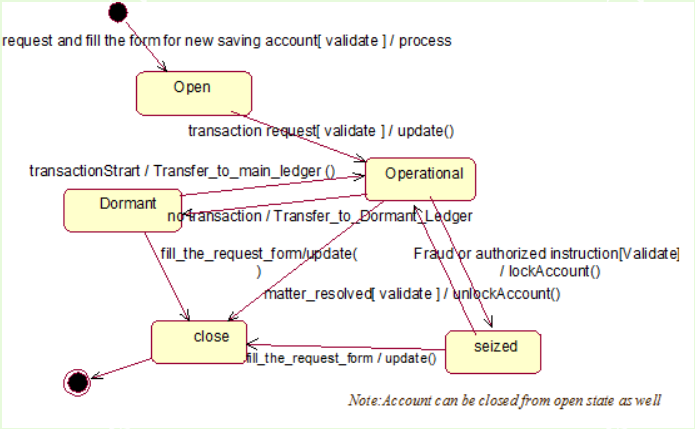

Figure 9.4: State Transition Diagram for Account Class

Note:

- A state diagram will not be created for every class.
- State diagrams are used only for those classes that exhibit interesting behavior.
- State diagrams are also useful to investigate the behavior of user interface and control classes.
- State diagram are used to show dynamics of a individual class

9.3. Activity Diagrams

It is a special kind of state diagram and is worked out at use case level. These are mainly targeted towards representing internal behavior of a use case. These may be thought as a kind of flowchart. Flowcharts are normally limited to sequential process;

activity diagrams can handle parallel process.

Activity diagrams are recommended in the following situations:

- Analyzing use case
- Dealing with multithreaded application
- Understanding workflow across many use cases.

They represent the performance of operations and transitions that are triggered. They captures dynamic behavior (Activity-Oriented).

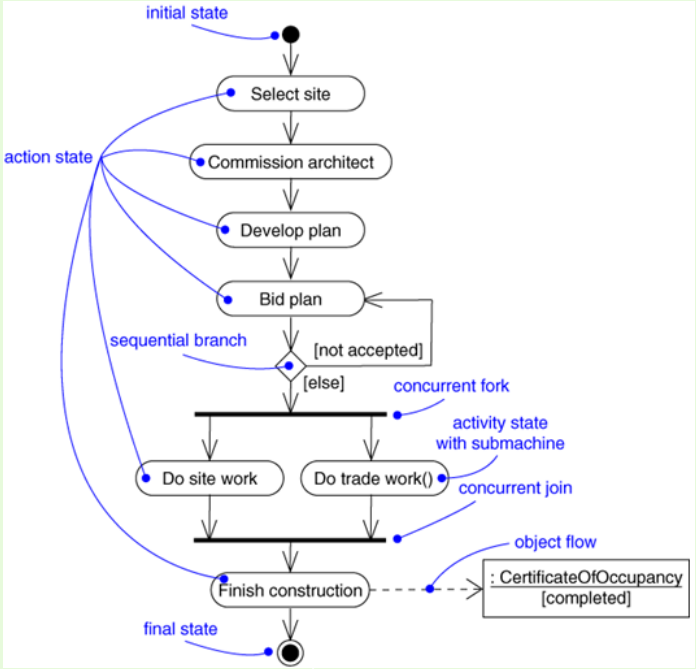

Figure 9.5: Activity Diagram

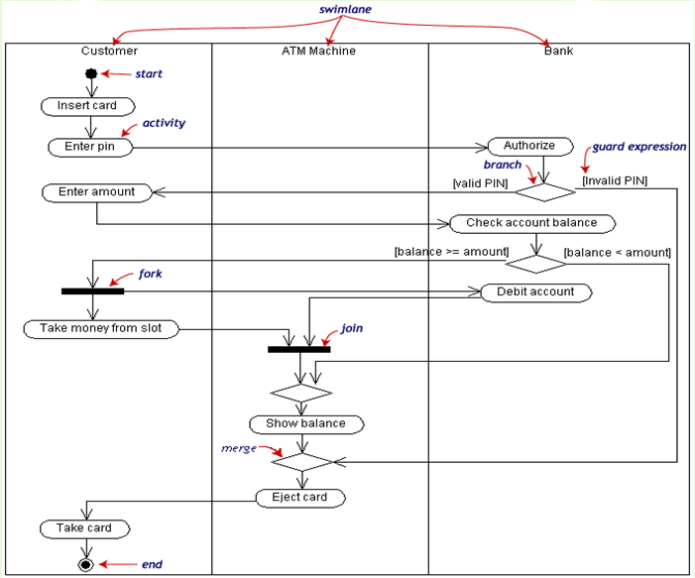

Figure 9.6: Activity Diagram

9.3.1. Consistency Checking

Consistency checking is the process of ensuring that, information in both static view of the system (class diagram) and the dynamic view of the system (sequence and collaboration diagram) is telling the same story.

9.4. Implementation Diagrams

Implementation diagrams show aspects of implementation, including source code structure and run-time implementation structure.

They come in two forms:

1. Component diagrams show the structure of the code itself and

2. Deployment diagrams show the structure of the run-time system.

9.5. Component Diagram

Component diagrams illustrate the organizations and dependencies among software components. A component diagram shows the dependencies among software components, including source code components, binary code components, and executable components.

A software module may be represented as a component type. Some components exist at compile time, some exist at link time, some exist at run time, and some exist at more than one time. A compile-only component is one that is only meaningful at compile time. The run-time component in this case would be

an executable program. A component diagram has only a type form, not an instance form. To show component instances, use a deployment diagram (possibly a degenerate one without nodes). A component may be

- A source code component
- A run time components
- An executable component
- Dependency relationship.

9.5.1. Notation

A component diagram is a graph of components connected by dependency relationships. Components may also be connected to components by physical containment representing composition relationships. A diagram containing component types and

node types may be used to show compiler dependencies, which are shown as dashed arrows (dependencies) from a client component to a supplier component that it depends on in some way.

The kinds of dependencies are language-specific and may be shown as stereotypes of the dependencies. The diagram may also be used to show interfaces and calling dependencies among components, using dashed arrows from components to interfaces on other components. Has only a type form, not an instance form.

To show component instances, use a deployment diagram (possibly a degenerate one without nodes).

9.5.2. Component Diagram for Withdrawal of Cash

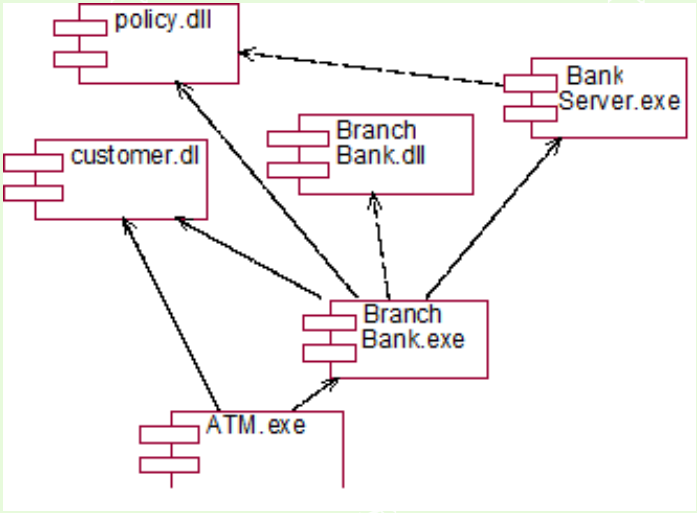

Figure 9.7: Cash Withdrawal Component Diagram

9.6. Deployment Diagram

Deployment diagrams show the configuration of run-time processing elements and the software components, processes, and objects that live on them. Software component instances represent run-time manifestations of code units. Components that do not exist as run-time entities (because they have been compiled away) do not appear on these diagrams, they should be shown on component diagrams. A deployment diagram shows the relationship among software and hardware components in the delivered system. These diagrams include nodes and connections between nodes. Each node in deployment diagram represents some kind of computational unit, in most cases a piece of hardware. Connection among nodes show the communication path over which the system will interact. The connections may represent direct

hardware coupling line RS-232 cable, Ethernet connection, they also may represent indirect coupling such as satellite to ground communication.

9.6.1. Notation

A deployment diagram is a graph of nodes connected by communication associations. Nodes may contain component instances. This indicates that the component lives or runs on the node. Components may contain objects, this indicates that the object is part of the component. Components are connected to other components by dashed-arrow dependencies (possibly through interfaces). This indicates that one component uses the services of another component. A stereotype may be used to indicate the precise dependency, if needed. The deployment type diagram

may also be used to show which components may run on which nodes, by using dashed arrows with the stereotype ■supports■. Migration of components from node to node or objects from component to component may be shown using the ■becomes■ stereotype of the dependency relationship. In this case the component or object is resident on its node or component only part of the entire time. Note that a process is just a special kind of object.

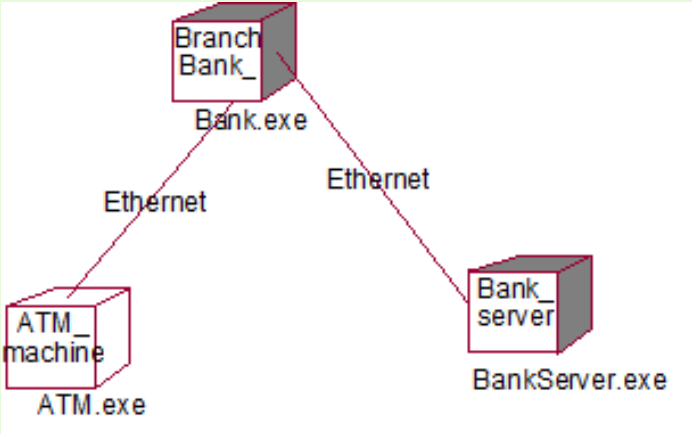

Figure 9.8: Deployment Diagram

9.7. Package Diagrams

Package diagrams are a valuable tool for grouping logically depending elements together.

For packaging, place those objects together which:

- are in the same subject area - closely related by concept or purpose,
- are in a type hierarchy,
- participate in the same use case,
- are strongly associated.

9.8. Summary

Learning Activities

Develop a simple sequence diagram in which: a) an object O receives message m from an actor; b) O creates a new object P; c) P sends a message to Q; d) P returns after receiving the response from Q; e) O destroys P

Revision Questions

Example ✎**.** Explain the difference between a state that has no outgoing transition at all, and one with an arrow into a stop (end) marker.

Solution: for revision □

EXERCISE 1. ✍ Explain the difference between component diagram and a deployment diagram.

EXERCISE 2. ✍ Describe the purpose of an activity diagram

References and Additional Reading Materials

1. Richard Lee , William M.Tepfenhart (2000). UML and C++ - A practical guide to object oriented development" , ISBN-13: 978-0130290403
2. John. W Satzinger (2004). Object Oriented Analysis and Design with the Unified Process. Cengage Learning, ISBN-13: 978-0619216436
3. Journal of computer science and Technology ISSN 1000-9000